

Milestone 1 — Data and Infrastructure Design

1. Raw data storage

The original motor insurance dataset and its corresponding data dictionary are stored in Google Cloud Storage.

The dataset is stored as a csv file and the description of the variable descriptors is stored as an xlsx file. The csv file contains 354,140 rows, delimited by semicolons, three years of data and repeated insured.

Bucket: `gs://setu-data-handling/`

Raw data location: `gs://setu-data-handling/raw/`

The original files are retained without modification so that processed datasets can be regenerated from the source data.

A local copy is also stored under `data/raw/` for development and is excluded from Git version control.

I considered storing the data only on the local filesystem. This would be sufficient for the current dataset size and provides fast access during development. I am therefore keeping a local copy under `data/raw/`.

I decided not to use a relational database at this stage because the project is mainly working with batch data rather than data that is being updated continuously. The dataset is loaded, processed and then used for modelling, so storing the files in object storage is enough for the current workflow. A database could still be useful later if the project needs more structured queries or regular updates, but it would add extra complexity for now.

I chose Google Cloud Storage as the main storage location because I may want to run later parts of the project outside my local machine, for example in Colab or in a cloud environment. Keeping the main copy in GCS means I can access the same raw and processed data from different places, while still keeping a local copy for quicker development.

2. Processed data storage and file formats

I plan to keep the processed datasets in the same Google Cloud Storage bucket as the raw data, but under a separate `processed/` folder:

`gs://setu-data-handling/processed/`

The processed data will be saved as Parquet files. I chose Parquet instead of CSV because it keeps column data types and is compressed, which should make it more convenient for repeatedly loading the data during analysis and model training.

The original CSV and Excel files will stay unchanged in `raw/`, while cleaned and model-ready versions will be written to `processed/`.

The processed files are stored under the paths:

processed/train_2022.parquet
processed/validation_2022.parquet
processed/test_2023.parquet
processed/future_2024.parquet

3. Database / object storage decision

Google Cloud Storage will be used for both the raw and processed project data. I decided not to use a relational database at this stage because the project is mainly working with batch data. The data is loaded, processed and then used for modelling, so object storage is enough for the current workflow.

This also fits with the file types being used in the project. The original data is stored as CSV and XLSX files, while the processed datasets are stored as Parquet files. These can be read directly when needed for modelling. A database could still be useful later if the project develops to the point where more structured querying or regular updates are needed, but for now it would add extra complexity without much benefit.

4. Data versioning

The original raw files are kept unchanged so that the processed datasets can always be recreated from the same source data.

Google Cloud Storage object versioning is enabled on the bucket, so if a file is overwritten an older version can still be recovered.

The code used to create the processed datasets is tracked in Git. If I make a major change later to the cleaning, feature construction or split, I will create a new processed-data version rather than treating it as the same dataset.

5. Data access

During development I'll keep a local copy of the raw data in data/raw/, which is excluded from Git using .gitignore. The same raw data is also stored in Google Cloud Storage under gs://setu-data-handling/raw/, while processed files are kept under gs://setu-data-handling/processed/. For local work I'm using project-relative paths in Python, for example:

```
data_path = Path("data/raw/Dataset of motor insurance portfolio.csv")
```

6. Data split / validation strategy

I am using a time-based split so that the model is tested on later data rather than on a random sample from the same period.

The split will be:

- 2022 — model development data
 - 80% training
 - 20% validation
- 2023 — out-of-time test set
- 2024 — future / production-style data kept for later monitoring and model updating

The 2022 data has 67,172 rows, so there is still enough data to create separate training and validation sets. The split will be reproducible using a fixed random seed.

One issue with the time split is that some insureds appear in more than one year. There are no repeated `insured_id` values within 2022, but an insured in the 2022 development data may also appear in the 2023 test set.

I am keeping these repeated insureds in the test set because returning customers are part of the real portfolio. However, I will also check model performance separately for insureds that were already present in 2022 and insureds that are new in 2023. This should show whether repeated customers are making the overall test performance look better than performance on genuinely unseen insureds.

There is also a large change in `business_type` across years, with much more portfolio business in 2023 and 2024 than in 2022. Because of this I also plan to compare performance for new business (`NB`) and portfolio business (`P`) separately. The full 2023 portfolio will still be the main test set.

Leakage controls

`insured_id` will be kept so I can track policies across years, but it will not be used as a model predictor.

Claim counts and incurred costs are outcomes, so these will not be included as input features. I will also only use variables that would realistically have been available when the policy was priced.

Any preprocessing that learns values from the data, such as missing-value imputation, scaling or encoding, will be fitted using the 2022 training set only and then applied to the validation and test data.

Some insureds appear in both the development and later test years. I am not removing these automatically because this reflects the real portfolio, but I will report test performance separately for previously seen and unseen insureds. I will also look separately at `NB` and `P` business because the mix of these changes quite a lot over time.

7. Feature description

The model will use information about the policy, driver, vehicle and location that would be available when the policy is being priced.

The main features I intend to use are:

Feature group	Variables	Description
Policy	<code>policy_type</code> , <code>business_type</code> , <code>payment_frequency</code> , <code>bonus_score</code>	General policy information and some information about previous claims/bonus history
Driver	<code>driver_age</code>	Age of the main driver
Vehicle	<code>vehicle_age</code> , <code>fuel_type</code> , <code>vehicle_value</code> , <code>seats</code> , <code>power_to_weight_ratio</code> , <code>vehicle_brand</code>	Information about the insured vehicle

Feature group	Variables	Description
Geography	<code>municipality_type</code> , <code>circulation_area</code>	Basic location information linked to the policy

This is not the final feature list yet. These will be reviewed and refined further after eda in milestone 2.

`total_exposure` is an interesting variable as it denotes how long the policy has actually been insured for. This will be used to model claim frequency rather than being used as a standard input variable.

The main outcomes I am interested in are:

- `total_claims` - number of claims during the exposure period
- `total_incurred` - total cost of the claims

The basic idea is to model how often claims happen and how much they cost, then combine these to get an estimate of expected claims cost.

There are also some variables I do not plan to use as predictors. `insured_id` is just an identifier, so I will keep it for tracking policies across years but not use it in the model. Claim and incurred-cost variables are outcomes, so they cannot be used as inputs.

I am also leaving the premium variables out for now because they already contain information from the insurer's existing pricing and I don't want the model to just learn the current price.

I also checked a couple of variables that were not completely clear from the names. `policy_status` shows whether the policy was active or cancelled at the time of analysis, so I am leaving this out because that information may not have been known when the policy was originally priced. `age_driving_licence` is actually the year the driver got their licence, so I will keep this and later convert it into something more useful such as years since the licence was obtained.

8. Data types and formats

The project uses structured tabular data. Each observation is represented as a row with a fixed set of named columns, which makes the dataset suitable for processing with pandas and standard tabular machine-learning tools.

The dataset contains a mixture of:

- **Numerical variables**, including continuous values such as `vehicle_value`, `total_exposure` and `total_incurred`, and discrete/count variables such as `total_claims` and `seats`.
- **Categorical variables**, including `policy_type`, `business_type`, `payment_frequency`, `bonus_score`, `fuel_type`, `vehicle_brand`, `municipality_type` and `circulation_area`.
- **Identifier/time variables**, including `insured_id` and `year`.

The source portfolio is supplied as a semicolon-delimited CSV file:

`Dataset of motor insurance portfolio.csv`

and is loaded using:

```
pd.read_csv(csv_path, sep=";")
```

9. Reproducibility of data collection

The project uses the public dataset:

“A detailed dataset of motor insurance policies with coverage-specific financial information”

Source: Mendeley Data

Licence: CC BY 4.0

The raw files used are:

- `Dataset of motor insurance portfolio.csv`
- `Descriptive of variables.xlsx`

The original files will be retained unchanged in the project’s raw-data storage location so that later processed datasets can always be regenerated from the same source files.

The portfolio CSV is semicolon-delimited.

10. Reproducibility of preprocessing

I plan to put the preprocessing into Python scripts rather than doing all of the cleaning manually in notebooks. The idea is that I should be able to start with the original CSV and recreate the same processed datasets again.

The rough preprocessing steps will be:

1. Load the original semicolon-separated CSV.
2. Check that the columns and data types look as expected.
3. Check for duplicate rows and duplicate `insured_id/year` combinations.
4. Keep the original raw files unchanged.
5. Remove fields that should not go into the model. `insured_id` will just be kept for tracking policies, while claim and incurred-cost fields will be treated as outcomes. I also plan to leave out the existing premium fields.
6. Only use predictors that would realistically have been known when the policy was priced.
7. Deal with missing values after the split. For example, if I use a median to fill in missing values, that median will be worked out from the training set only and then reused for the other datasets.
8. Handle `total_exposure` separately, since it tells me how long the policy was actually at risk rather than being a normal predictor.
9. Encode categorical variables using rules fitted on the training data.
10. Split the data into 2022 train/validation, 2023 test and 2024 future data.
11. Save the processed datasets as Parquet files in the versioned `processed/` folder in GCS.
12. Save some basic metadata with each processed version, such as the raw data version, row counts, processing date and Git commit.

Anything learned from the data during preprocessing, such as imputation values or category encodings, will be fitted using the training data only and then reused for the validation, test and future datasets.

There are still a couple of cleaning rules I need to settle during the next stage, especially how to deal with zero-exposure rows and claims with zero incurred cost. Once these are decided they will be added to the pipeline rather than handled manually.